TASKFLOW-API

API de Gerenciamento de Tarefas

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	TASKFLOW-API — API de Gerenciamento de Tarefas
Gerente de Projeto	Victhor Hugo Wandersee Rodrigues Philippe
Gerente/Responsável de GCS/DevOps	Victhor Hugo Wandersee Rodrigues Philippe
Fábrica de Software / Squad	Trabalho individual — atuando em todos os papéis de GCS/DevOps
Repositório principal	https://github.com/DevVHW/taskflow-api

Histórico de Revisões

Data	Versão	Descrição	Autor
22/09/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD), a partir do projeto exemplo TaskFlow API.	Victhor Hugo Wandersee Rodrigues

Data	Versão	Descrição	Autor
			Philippe

Sumário

(No Word, clique com o botão direito sobre esta área e escolha "Atualizar campo" — ou pressione Ctrl+A e F9 — para gerar o sumário automaticamente a partir dos títulos do documento.)

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) executadas ao longo do ciclo de vida do projeto TaskFlow API, com ênfase em práticas DevOps — Integração Contínua (CI), Entrega/Implantação Contínua (CD) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer o padrão a ser seguido no desenvolvimento e na operação do TaskFlow API, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por um pipeline automatizado de build, teste, integração e implantação executado no GitHub Actions.

1.2 Escopo

Este PGC é destinado ao autor do projeto, atuando individualmente como responsável por todos os papéis de desenvolvimento e operação do sistema TaskFlow API — uma API REST de gerenciamento de tarefas (CRUD de tarefas), utilizada aqui como projeto de estudo para aplicação prática de Gestão de Configuração de Software com enfoque DevOps. O escopo abrange o código-fonte da aplicação, os scripts de pipeline (CI/CD), o Dockerfile e a documentação técnica do repositório único do projeto.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
GHCR	GitHub Container Registry — registro de imagens Docker do projeto
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
GitHub Flow	Estratégia de branching simplificada baseada em branches de curta duração e PRs para a main
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - ex.: 1.0.0
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado, enfoque DevOps.
- Repositório do projeto: https://github.com/DevVHW/taskflow-api
- Documentação oficial do GitHub Actions: https://docs.github.com/actions

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente pelo GitHub Actions gera evidências de rastreabilidade (logs de execução do workflow, tags de imagem versionadas no GHCR, histórico de Pull Requests), reduzindo o esforço manual e o tempo entre a mudança e sua entrega.

2. Cultura DevOps e Papéis na Gerência de Configuração

Por se tratar de um projeto individual, todos os papéis abaixo são exercidos pelo autor do projeto. Ainda assim, os papéis são descritos separadamente para deixar explícitas as responsabilidades de cada função dentro do ciclo de vida do produto, conforme a prática "você constrói, você executa" (you build it, you run it).

Papel	Responsabilidades (exercidas pelo autor do projeto)
Gerente de Projeto / Product Owner	Prioriza as funcionalidades da API (tarefas), define o escopo de cada entrega e acompanha os indicadores de pipeline (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os workflows de CI/CD (ci.yml e cd.yml), define a estratégia de branching/versionamento e administra o GitHub Container Registry.
Desenvolvedor(a)	Implementa as rotas da API (src/app.js), escreve os testes automatizados (Jest/Supertest) e integra o código via Pull Request.
SRE / Operações	Acompanha a execução dos workflows de deploy, define os health checks (rota /health) e realiza rollback via redeploy de uma tag anterior da imagem.
QA / Automação de Testes	Mantém a suíte de testes automatizados executada a cada push/PR pelo job de CI.
Segurança (DevSecOps)	Mantém o Dependabot habilitado no repositório para análise de dependências (SCA) e revisa alertas de segurança do GitHub.
Comitê de Controle de Mudanças (CCM)	Não aplicável de forma colegiada neste projeto individual; mudanças de alto risco (ex.: alteração de contrato da API) são registradas via issue antes da branch de feature, substituindo a aprovação formal do comitê.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas no projeto TaskFlow API:

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento de código-fonte, workflows e documentação como código
Orquestração de CI/CD	GitHub Actions	Automação de build, lint, testes e publicação/implantação
Empacotamento	Docker	Geração de imagem versionada e imutável da aplicação

Categoria	Ferramenta(s) adotada(s)	Finalidade
Registro de artefatos	GitHub Container Registry (GHCR)	Armazenamento e versionamento das imagens Docker publicadas pelo CD
Testes	Jest + Supertest	Testes unitários e de integração da API
Análise estática	ESLint	Padronização e verificação de qualidade do código JavaScript
Gestão de dependências / SCA	Dependabot (GitHub)	Identificação e proposta de correção de vulnerabilidades em dependências
Rastreamento de tarefas	GitHub Issues	Vinculação de commits/PRs aos itens de trabalho do projeto

Terraform/Ansible, Kubernetes e ferramentas de observabilidade dedicadas (ex.: Prometheus/Grafana) não são utilizados neste projeto de estudo por sua simplicidade — o item 7 (Infraestrutura como Código) descreve como esses conceitos são aplicados de forma simplificada por meio do Dockerfile e dos "environments" do GitHub Actions.

3.2 Ambientes e Infraestrutura

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento (local)	Execução e testes do código na máquina do autor durante o desenvolvimento (npm start).	Manual, a critério do desenvolvedor
CI (efêmero)	Ambiente descartável do runner do GitHub Actions, onde o código é buildado e testado a cada Pull Request.	Abertura/atualização de Pull Request ou push na main
Homologação / Staging	Validação da imagem publicada antes da promoção final, com execução de smoke tests simulados.	Publicação bem-sucedida da imagem no GHCR (job publish-image)
Produção	Ambiente utilizado pelos usuários finais da API.	Aprovação do "environment" production do GitHub Actions, configurado como gate manual

Os ambientes de staging e produção estão representados no workflow de CD por meio dos "GitHub Environments", que podem ser configurados com regras de proteção (ex.: aprovação manual obrigatória) diretamente nas configurações do repositório, sem custo adicional de infraestrutura para fins deste projeto de estudo.

3.3 Estratégia de Branching e Versionamento

Adota-se GitHub Flow, com branches de vida curta, adequado ao ritmo de um projeto individual:

- main — sempre implantável; protegida por regra de Pull Request obrigatório e pipeline de CI verde.
- feature/<descrição-curta> — branch de curta duração para cada nova funcionalidade ou correção (ex.: feature/patch-marcar-tarefa-concluida).
- hotfix/<descrição> — correções emergenciais aplicadas diretamente sobre produção e reintegradas à main.

Versionamento: adoção de Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor...), permitindo rastrear a evolução do projeto a partir do histórico de commits.

4. Identificação da Configuração

4.1 Itens de Configuração

Os seguintes artefatos são itens de configuração formais do projeto, versionados no repositório único do TaskFlow API:

- Código-fonte da aplicação (src/) e dos testes automatizados (__tests__/)
- Definições de pipeline como código (.github/workflows/ci.yml e .github/workflows/cd.yml)
- Dockerfile e .dockerignore
- Arquivo de dependências travado (package-lock.json)
- Arquivo de configuração de lint (.eslintrc.json)
- Documentação técnica (README.md) e este PGCS

Não há scripts de migração de banco de dados nem arquivos de configuração sensíveis (segredos) neste projeto, pois a API utiliza armazenamento em memória para fins didáticos.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<descrição-curta>	feature/endpoint-remover-tarefa
Commit	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(tasks): adiciona endpoint de remoção de tarefa
Tag de release	Versionamento Semântico vX.Y.Z	v1.0.0
Imagem de container	ghcr.io/<usuário>/taskflow-api:<versão|sha>	ghcr.io/devvhw/taskflow-api:a1b2c3d

4.3 Estrutura do Repositório

Estrutura de diretórios do repositório único do projeto:

- /src — código-fonte da aplicação (app.js, index.js)
- /__tests__ — testes automatizados (Jest + Supertest)
- /.github/workflows — definições dos pipelines de CI e CD como código
- Dockerfile, .dockerignore — empacotamento da aplicação
- README.md, PGCS.pdf — documentação do projeto

5. Integração Contínua (CI)

Todo Pull Request para a branch main dispara automaticamente o workflow ci.yml. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso.

5.1 Etapas do Pipeline de CI

1. Checkout do código (actions/checkout).
2. Configuração do Node.js 20 com cache de dependências npm.
3. Instalação das dependências (npm ci).
4. Análise estática de código com ESLint (npm run lint).
5. Testes unitários e de integração com medição de cobertura (npm test, via Jest).

6. Build de validação da imagem Docker (docker build), garantindo que a imagem publicável pelo CD é sempre buildável.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige que o workflow de CI seja concluído com sucesso antes do merge (quality gate).
- Por se tratar de projeto individual, a autorrevisão do diff do PR substitui a revisão por pares, mantendo o registro formal da mudança no histórico do GitHub.
- PRs devem ser pequenos e de vida curta, para facilitar a leitura do diff e reduzir o risco de regressões.

5.3 Critérios de Qualidade (Quality Gate)

- Zero erros de lint bloqueante (ESLint deve encerrar com código de saída 0).
- 100% dos testes automatizados passando.
- Build da imagem Docker concluído sem erros.

6. Entrega e Implantação Contínua (CD)

Após a publicação da imagem pelo job publish-image do workflow cd.yml, o mesmo workflow promove essa versão, sem recompilação, através dos ambientes de staging e produção.

6.1 Etapas do Pipeline de CD

7. Login no GitHub Container Registry (GHCR).
8. Build e push da imagem versionada, com tags pelo SHA do commit e latest.
9. Deploy em ambiente de Homologação/Staging (representado pelo "environment" staging).
10. Execução de smoke tests em staging.
11. Gate de aprovação para produção — "environment" production configurável com aprovação manual no GitHub.
12. Deploy em Produção.
13. Geração da tag de release (baseline) correspondente ao SHA implantado.

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Estratégia de referência para uma futura implantação em um orquestrador de containers (ex.: substituição gradual das instâncias).
Recriação simples	Estratégia efetivamente usada neste projeto de estudo: a nova imagem publicada substitui diretamente a anterior no ambiente de destino, dado o baixo risco/criticidade do projeto.
Rollback por tag	Em caso de falha, o ambiente pode ser reimplantado a partir de uma tag de imagem anterior armazenada no GHCR.

6.3 Rollback e Recuperação

Toda implantação deve ser reversível. O GHCR mantém as imagens de versões anteriores publicadas pelo pipeline, permitindo que, em caso de falha nos smoke tests ou nos health checks (rota /health) pós-deploy, a implantação seja revertida manualmente para a tag anterior.

7. Infraestrutura como Código (IaC)

Dada a simplicidade do projeto, a IaC é aplicada de forma simplificada: o Dockerfile e o .dockerignore descrevem, como código versionado, o ambiente de execução da aplicação (imagem base, dependências, comando de start), garantindo paridade entre os ambientes de desenvolvimento, staging e produção.

7.1 Versionamento de Infraestrutura

- Alterações no Dockerfile seguem o mesmo processo de PR + pipeline de CI usado para o código da aplicação.
- Os "GitHub Environments" (staging/production) e suas regras de proteção são configurados pela interface do GitHub e documentados neste PGCS, funcionando como a definição de infraestrutura de promoção do pipeline.

7.2 Ambientes Efêmeros

O próprio ambiente de execução do job de CI no GitHub Actions é, por definição, efêmero: um runner novo é provisionado a cada execução e destruído ao final, sem persistência entre execuções.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (novas funcionalidades, correções) seguem o fluxo padrão: abertura de issue no GitHub → branch de feature → Pull Request → pipeline de CI → merge → pipeline de CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

Por se tratar de projeto individual, não há um CCM formal. Mudanças de maior impacto — como alteração no contrato de uma rota existente da API — são precedidas da abertura de uma issue descrevendo o motivo e o impacto da mudança, antes da criação da branch de feature, substituindo a aprovação colegiada por um registro documentado e revisável.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Cada implantação bem-sucedida em produção gera uma baseline: a tag de imagem publicada no GHCR (identificada pelo SHA do commit), associada ao commit correspondente na branch main.

9.2 Notas de Versão (Release Notes)

As notas de release podem ser geradas a partir do histórico de Conventional Commits desde a última tag e publicadas manualmente como uma Release no GitHub a cada versão relevante (ex.: v1.0.0).

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

O monitoramento neste projeto de estudo é feito de forma simplificada por meio da rota /health (health check) e dos logs de execução disponíveis diretamente na aba Actions do GitHub, que registram cada etapa dos pipelines de CI e CD.

10.2 Métricas DORA

Métrica	Descrição	Como é observada no projeto
Deployment Frequency	Frequência com que o código é implantado em produção.	Contagem de execuções bem-sucedidas do job deploy-production no histórico do Actions.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.	Diferença entre o timestamp do commit e a conclusão do job deploy-production.
Change Failure Rate	Percentual de implantações que resultam em falha.	Proporção de execuções do workflow CD marcadas como "failed" no Actions.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.	Tempo entre a detecção da falha e a conclusão de um rollback por tag anterior.

10.3 Relatórios e Auditorias

O histórico de Pull Requests, commits e execuções de workflow (aba Actions) do repositório GitHub constitui o principal insumo de auditoria de GCS deste projeto, dispensando relatórios manuais.

11. Marcos

- Criação do repositório e estrutura inicial do projeto TaskFlow API.
- Implantação do pipeline de CI (lint + testes + build de validação).
- Implantação do pipeline de CD (publicação da imagem no GHCR e deploy simulado por ambientes).
- Primeira release automatizada (v1.0.0) com geração de baseline.

12. Treinamento e Recursos

Para manter as práticas de CI/CD descritas neste plano, são necessários: conhecimento básico de Git/GitHub, sintaxe de workflows do GitHub Actions (YAML), Docker e o próprio stack da aplicação (Node.js/Express). A documentação oficial do GitHub Actions e do Docker é suficiente como material de referência para a manutenção do pipeline.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

As bibliotecas de terceiros utilizadas (Express, Jest, Supertest, ESLint) são itens de configuração versionados explicitamente no package-lock.json (lockfile).

- O Dependabot, habilitado no repositório GitHub, realiza análise automática de vulnerabilidades em dependências (SCA).
- Atualizações de dependências propostas pelo Dependabot são submetidas ao mesmo fluxo de Pull Request e pipeline de CI das demais mudanças.

Não há subcontratados ou fornecedores externos de software neste projeto individual.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção, no projeto TaskFlow API:

Etapa	Descrição	Gatilho
CI	Lint, testes unitários/integração, build de validação da imagem Docker.	Abertura/atualização de Pull Request ou push na main
Merge	Integração da mudança na branch main.	CI verde
CD — Publicação	Build e push da imagem versionada para o GHCR.	Push na branch main (pós-merge)
CD — Staging	Deploy simulado em homologação + smoke tests.	Publicação da imagem concluída
Gate de Release	Aprovação manual do "environment" production no GitHub Actions.	Sucesso do deploy em staging
CD — Produção	Deploy simulado em produção e geração da baseline/tag.	Aprovação do gate de release

Nota: este documento foi elaborado a partir do template institucional de PGCS com enfoque DevOps, adaptado ao contexto real do projeto TaskFlow API (ferramentas efetivamente utilizadas e criticidade de um projeto de estudo individual), mantendo os quatro pilares da GCS — identificação, controle de versão, controle de mudanças e auditoria/status — agora suportados por automação via GitHub Actions.